| | | | |
|---|---|---|---|
| **Name of Student:**<br>VIKAS FOOLCHAND KUSHWAHA | | | |
| **Roll Number:**<br>29/B | | **Lab Practical Number:**<br>05 | |
| **Title of LAB Practical:**<br>Implementation of File Handling | | | |
| **DOP:**<br>08/11/2026 | | **DOS:**<br>12/11/2026 | |
| **CO Mapped:**<br>CO1 | **PO Mapped:**<br>PO3, PO5, PSO1, PSO2 | **Faculty Signature:** | **Marks:** |

# **Aim**: Implementation of File Handling

# **Description:**

File Handling means performing file system operations such as **read, write, append, delete,** etc. NodeJS has a builtin module called fs, an abbreviation of the word 'filesystem' which helps us to perform such operations. It provides methods to execute the operations synchronously as well as asynchronously.

**Key Features of File Handling in Node.js:**

1. **Built-in Module:** Doesn't require installation of any external libraries.
2. **Synchronous and Asynchronous:** Supports asynchronous execution using callbacks or promises for non-blocking operations. Synchronous methods block the execution until the operation is complete.
3. **Cross-platform compatibility:** Provides interoperability with various different operating systems.

**Common File Operations:**

1. **Create:** Creates a new file or overrides the content of existing file.

   **Synchronous method:** fs.writeFile()

   **Asynchronous method:** fs.writeFileSync()

2. **Append:** Appends new content to the end of file instead of overwriting.

   **Synchronous method:** fs.appendFile()

   **Asynchronous method:** fs.appendFileSync()

3. **Read:** Opens a file for reading and gets the content.

   **Synchronous method:** fs.readFile()

   **Asynchronous method:** fs.readFileSync()

4. **Delete:** Deletes a file from the filesystem.

   **Synchronous method:** fs.unlink()

5. **Exists:** Checks if a file exists on the file system.

   **Synchronous method:** fs.existSync()

**Applications of File Handling:**

A. **Data Storage:** Storing logs, configurations, and user data.
B. **File-Based Databases:** Working with JSON, CSV, or other structured files.
C. **Static Content Delivery:** Serving files like HTML, CSS, and images in web applications.
D. **Temporary File Management:** Creating and cleaning up temporary files during application runtime.

# Code:

**QUESTION**:
**Using File Handling, demonstrate all basic file operations (Create, write, read, delete).**

**CODE**

```
const fs = require('fs');

// Set path to our file
const filePath = './example.txt';

// 1. Create and write a file
fs.writeFile(filePath, 'This is the initial content of the file.\n',
(err) => {
  if (err) {
          return  console.error('Error  creating/writing  file\n',
err.message);
```

**CODE**

```
  }
  console.log(':: File created and written successfully.');

  // 2. Append to the file
  fs.appendFile(filePath, 'Appended text to the file.', (err) => {
    if (err) {
            return  console.error('Error  appending  to  the  file\n',
err.message);
    }
    console.log(':: Content appended successfully.');

    // 3. Read the file
    fs.readFile(filePath, 'utf8', (err, data) => {
      if (err) {
        return console.error('Error reading file\n', err.message);
      }
      console.log(':: --- Beginning of file content ---');
      console.log(data)
      console.log(':: --- End of file content ---');

      // 4. Delete the file
      fs.unlink(filePath, (err) => {
        if (err) {
          return console.error('Error deleting file\n', err.message);
        }
        console.log(':: File deleted successfully.');
      });
    });
  });
});
```

# Output:

```
> node filehandling.js
:: File created and written successfully.
:: Content appended successfully.
:: --- Begining of file content ---
This is the initial content of the file.
Appended text to the file.
:: --- End of file content ---
:: File deleted successfully.
```

# Conclusion:

In this practical, we learned about the various file system operations like reading, writing, appending, deleting, etc, that can be performed in Node.js using the 'fs' file system module. It demonstrates the efficiency of Node.js when performing I/O operations in asynchronous manner.